

DSM2: Distilled Sequence Model v2

Technical Report of the Training Pipeline

GleghornLab

March 2, 2026

Contents

1 Introduction

DSM2 (Distilled Sequence Model v2) is a self-supervised protein language model (pLM) trained via knowledge distillation from a frozen, large-scale teacher pLM. The student is an E1-architecture transformer that learns to (i) reconstruct masked amino-acid tokens (masked language modelling, MLM), (ii) match the intermediate hidden-state representations of the teacher on a per-token basis (JEPA loss), and (iii) preserve the intra-batch relational geometry of the teacher’s representations at the sequence level (contrastive loss).

The training pipeline has three sequential loss phases, an exponential-moving-average (EMA) teacher that eventually replaces the external teacher, and supports both standard AdamW and the MuonClip optimizer (Muon + QK-Clip). Distributed data-parallel (DDP) training is supported out of the box.

The default teacher is `Synthyra/DPLM-3B`; the default student has a hidden dimension of 256, trained for 100 000 optimiser steps on `Synthyra/uniref50`.

2 Model Architecture

2.1 Teacher Model

The teacher is a frozen, pre-trained protein language model loaded in `bfloat16` and moved to the target device. All teacher parameters have `requires_grad = False`; the teacher is kept in `eval()` mode throughout training. The teacher is selected by inspecting the model path string:

Path substring	Class loaded
<code>dplm2</code>	<code>DPLM2Model</code>
<code>dplm</code>	<code>DPLMModel</code>
<code>esm2, fastesm</code>	<code>FastEsmModel</code>
<code>e1</code>	<code>E1ForMaskedLM</code>
(default)	<code>ESMplusplusModel</code>

The teacher exposes `config.num_hidden_layers` (number of transformer layers) and `config.hidden_size` (d_T , teacher hidden dimension), both of which are used to configure the student.

2.2 Student Model (DSM2 / E1 Backbone)

The student model is an instance of `DSM2`, a subclass of `E1ForMaskedLM` augmented with distillation infrastructure. The E1 backbone is a decoder-style transformer with pre- and post-attention RMSNorm, grouped-query attention (GQA), and a SwiGLU (or standard) MLP.

2.2.1 Configuration (DSM2Config)

`DSM2Config` extends `E1Config` with two additional fields:

- `teacher_hidden_size` (d_T): hidden dimension of the teacher (default 768).
- `use_teacher_projections`: whether to attach a linear projection per layer that maps student representations into teacher space (default `True`; disabled after the EMA teacher takes over).

The student is initialised from scratch when `pretrained_weights` is `None`; the following derived quantities are computed from the CLI arguments:

$d_s = \text{student_hidden_size}$ (must satisfy $d_s \bmod 64 = 0$)
 $d_{\text{ff}} = \lfloor d_s \times \text{student_expansion_ratio} \rfloor$
 $H = d_s / 64$ (number of attention heads, also number of KV heads)
 $N = \text{teacher.config.num_hidden_layers}$ (shared with teacher)

2.2.2 E1 Transformer Backbone

Each of the N transformer layers consists of:

1. **Pre-attention RMSNorm** applied to the residual stream.
2. **Grouped-query self-attention** with rotary positional embeddings (see Section ??).
 Projections: $W_Q \in \mathbb{R}^{d_s \times (H \cdot h_d)}$, $W_K \in \mathbb{R}^{d_s \times (H_k \cdot h_d)}$, $W_V \in \mathbb{R}^{d_s \times (H_k \cdot h_d)}$, $W_O \in \mathbb{R}^{(H \cdot h_d) \times d_s}$,
 all without bias, where $h_d = d_s / H$ is the per-head dimension.
3. **Post-attention RMSNorm** added to the residual before the MLP.
4. **MLP** (SwiGLU variant by default):

$$\text{FFN}(x) = W_2(\sigma(W_1 x) \odot W_3 x), \quad \sigma = \text{SiLU} \quad (1)$$

where $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_s}$ and $W_2 \in \mathbb{R}^{d_s \times d_{\text{ff}}}$. The standard (non-gated) MLP applies $W_2(\sigma(W_1 x))$ without a gate branch.

The backbone concludes with a final **RMSNorm** applied to the last hidden state.

2.2.3 MLM Head

The masked-language-modelling head converts the final hidden state into logits over the vocabulary:

$$\text{MLMHead}(h) = W_{\text{out}}(\text{LayerNorm}(\text{GELU}(W_{\text{in}} h + b_{\text{in}}))) + b_{\text{out}} \quad (2)$$

where $W_{\text{in}} \in \mathbb{R}^{d_s \times d_s}$, $W_{\text{out}} \in \mathbb{R}^{V \times d_s}$, and V is the vocabulary size.

2.2.4 Teacher Projection Layers

When `use_teacher_projections = True`, the student has N additional linear layers, one per transformer layer:

$$\phi^{(\ell)} : \mathbb{R}^{d_s} \rightarrow \mathbb{R}^{d_T}, \quad \phi^{(\ell)}(h) = W_{\phi}^{(\ell)} h \quad (3)$$

These projections are used to match the dimensionality of the student's hidden states to the teacher's before computing the JEPA and contrastive losses. They are removed from the model (and from the optimiser's parameter groups) when the EMA teacher becomes active, since the EMA teacher already operates in student-space $\mathbb{R}^{d_s}$.

2.3 Attention Backends

The attention computation is selectable via `attn_backend`:

Backend	Implementation
<code>kernels_flash</code>	Flash-Attention 3/2 via the <code>kernels</code> package
<code>flash_attn</code>	Flash-Attention 2 via the <code>flash_attn</code> package
<code>flex</code>	PyTorch <code>FlexAttention</code> with sliding-window + dilation mask
<code>sdpa</code>	PyTorch <code>scaled_dot_product_attention</code>
<code>auto</code>	Resolved at runtime to the best available backend

The `flex` backend applies a local sliding-window attention mask of size `sliding_window_size` combined with a dilated global attention pattern with factor `dilation`, enabling sub-quadratic attention on long sequences.

3 Tokenizer and Positional Encoding

3.1 Tokenizer

The tokenizer is loaded from `tokenizer.json` (sourced from Synthra/Profluent-E1-150M). It is an amino-acid tokenizer with special tokens including `<bos>`, `<eos>`, `<pad>`, and a mask token (the `?` character). Boundary tokens 1 and 2 are used to delimit multi-chain inputs.

3.2 Dual Rotary Positional Embedding

Each token in the input carries two sets of position indices:

- **Within-sequence position IDs** (`within_seq_position_ids`): the position of a token within its own protein chain, up to `max_num_positions_within_seq` = 8192.
- **Global position IDs** (`global_position_ids`): the position of a token across the full packed context window.

Rotary embeddings are applied at each attention layer. The within-sequence RoPE uses base $\theta_{\text{local}} = 10\,000$, while the global RoPE uses $\theta_{\text{global}} = 100\,000$. Layers alternate between the two depending on `global_attention_every_n_layers`: when this parameter is 0 (the default for DSM2), all layers use within-sequence RoPE.

For an attention head dimension h_d , the rotary embedding for a position p is the block-diagonal matrix $R(p)$ with 2×2 rotation blocks:

$$R(p)_{2k,2k} = \cos(p\theta^{-2k/h_d}), \quad R(p)_{2k,2k+1} = -\sin(p\theta^{-2k/h_d}) \quad (4)$$

for $k = 0, \dots, h_d/2 - 1$, applied to both query and key vectors before the dot-product.

3.3 Multi-Chain Packing

The `E1BatchPreparer` utility handles multi-sequence inputs (e.g. comma-separated chains). Up to `max_num_sequences` = 512 chains may be packed into one context window, each assigned a unique sequence-type embedding from `embed_seq_id` $\in \mathbb{R}^{512 \times d_s}$ that is added to the token embedding before the first layer.

4 Data Pipeline

4.1 Dataset

By default, the model trains on Synthra/uniref50, a HuggingFace dataset providing `train`, `valid`, and `test` splits of UniRef50 protein sequences. The dataset is loaded via `datasets.load_dataset`,

shuffled with a fixed `shuffle_seed`, and optionally truncated to `train_limit`, `valid_limit`, and `test_limit` samples respectively.

Each sample is a raw amino-acid string from the column specified by `sequence_column` (default "sequence"). Sequences are tokenised and padded/truncated to at most `max_length = 2048` tokens by the `SequenceCollator`.

4.2 Batch Hierarchy: Patches and Patch Groups

DSM2 uses a two-level batching scheme:

Level	Parameter	Default
Micro-batch (<i>patch</i>)	<code>patch_size</code>	16 sequences
Patch group (per grad-accum step)	<code>patch_accum = ⌊batch_size/patch_size⌋</code>	4 patches
Gradient accumulation	<code>grad_accum</code>	4 steps
Effective batch per optimiser step (single GPU)	<code>batch_size × grad_accum</code>	256 sequences

The `DataLoader` emits micro-batches of size `patch_size`. The trainer collects `patch_accum` consecutive micro-batches into a *patch group*, computes the combined loss (accumulating gradients inside `model.no_sync()` where applicable), and repeats for `grad_accum` patch groups before calling `optimizer.step()`.

The effective number of sequences seen per optimiser step across all distributed ranks is:

$$N_{\text{eff}} = \text{patch_size} \times \text{patch_accum} \times \text{grad_accum} \times \text{world_size} \quad (5)$$

4.3 Distributed Sampling

In multi-GPU runs (`world_size > 1`), the training `DataLoader` uses `DistributedSampler` with `drop_last=True`. Validation and test loaders also use `DistributedSampler` with `drop_last=False` and no shuffling. In single-GPU runs, `RandomSampler` is used for all splits.

On Windows, the number of workers for evaluation loaders is forced to 0 to avoid expensive spawn latency; on Linux, `eval_num_workers` matches the training worker count.

5 Training Objective

5.1 Stochastic Masking

For each sequence i in a micro-batch of size B , a per-sample mask rate is drawn:

$$t_i \sim \text{Uniform}(\varepsilon, 1), \quad \varepsilon = 10^{-3} \quad (6)$$

During evaluation the mask rate is fixed:

$$t_i = 0.15 \quad \forall i \quad (7)$$

A token at position j (length L) is replaced by the mask token [MASK] with probability t_i , subject to the constraint that special tokens (BOS, EOS, PAD, boundary tokens) and padding positions are never masked:

$$m_{ij} = \mathbf{1}[\text{Uniform}(0, 1) < t_i] \cdot \mathbf{1}[x_{ij} \notin \mathcal{S}] \cdot \text{attn}_{ij} \quad (8)$$

where $\mathcal{S}$ is the set of special token IDs and attn_{ij} is the attention mask. The noisy input is $\hat{x}_{ij} = [\text{MASK}]$ if $m_{ij} = 1$, otherwise $\hat{x}_{ij} = x_{ij}$.

The teacher always receives the *original, unmasked* tokens x_i , while the student always receives the masked tokens $\hat{x}_i$.

Two derived masks are computed from m_{ij} :

$$\mathcal{M} = \{(i, j) : m_{ij} = 1\} \quad (\text{masked positions, CE targets}) \quad (9)$$

$$\mathcal{D} = \{(i, j) : m_{ij} = 0, \text{attn}_{ij} = 1, x_{ij} \notin \mathcal{S}\} \quad (\text{unmasked non-special positions, JEPA targets}) \quad (10)$$

If $\mathcal{D}$ is empty after masking (edge case), $\mathcal{D}$ falls back to all valid attention-mask positions. Similarly, if $\mathcal{M}$ is empty, CE falls back to all valid attention-mask positions.

5.2 Importance-Weighted Cross-Entropy Loss

The student’s logits $P_\theta(x_{ij} \mid \hat{x}_i)$ are computed by the MLM head over all positions. Cross-entropy is evaluated only at masked positions and re-weighted by the inverse of the sample’s mask rate (importance weight $1/t_i$) to correct for the varying number of masked tokens per sequence:

$$\mathcal{L}_{\text{CE}} = \frac{1}{B \cdot L} \sum_{(i,j) \in \mathcal{M}} \frac{-\log P_\theta(x_{ij} \mid \hat{x}_i)}{t_i} \quad (11)$$

This importance weighting makes the expected gradient contribution of each sequence equal regardless of how many tokens happen to be masked in that sample (a lower t_i means fewer masked tokens but each is up-weighted by $1/t_i$, and vice versa).

5.3 JEPA Distillation Loss

The Joint Embedding Predictive Architecture (JEPA) loss encourages the student’s intermediate representations—after projection into teacher space—to match the teacher’s representations at the unmasked context positions $\mathcal{D}$.

Let $\hat{h}_{ij}^{(\ell)} = \phi^{(\ell)}(h_{s,ij}^{(\ell)}) \in \mathbb{R}^{d_T}$ be the ℓ -th projected student hidden state at token (i, j) , and let $h_{t,ij}^{(\ell)} \in \mathbb{R}^{d_T}$ be the corresponding teacher hidden state.

The loss applies *depth weighting* $w_\ell = \ell/N$ that linearly increases with layer depth, emphasising higher-level semantic representations:

$$\mathcal{L}_{\text{JEPA}} = \frac{1}{|\mathcal{D}| \cdot N} \sum_{\ell=1}^N \frac{\ell}{N} \sum_{(i,j) \in \mathcal{D}} \frac{1}{d_T} \|\hat{h}_{ij}^{(\ell)} - h_{t,ij}^{(\ell)}\|_2^2 \quad (12)$$

The factor $1/d_T$ comes from averaging the squared difference over the hidden dimension before summing. The denominator $|\mathcal{D}| \cdot N$ normalises jointly over valid token positions and layers. All computations are performed in `float32` for numerical stability.

5.4 Contrastive Representation Loss

The contrastive loss aligns the *relational structure* of student and teacher representations across the batch. Instead of matching token-level representations directly, it matches the self-similarity matrices of pooled sequence representations at each layer.

Step 1: Mean-variance pooling. For each layer ℓ and sample i , the student and teacher hidden states are pooled over valid (attended) token positions into a $2d_T$ -dimensional vector:

$$\bar{s}_i^{(\ell)} = [\mu(\hat{H}_i^{(\ell)}); \sigma^2(\hat{H}_i^{(\ell)})] \in \mathbb{R}^{2d_T} \quad (13)$$

where μ and σ^2 are the masked mean and variance over the sequence dimension. This yields tensors $\bar{s}^{(\ell)}, \bar{t}^{(\ell)} \in \mathbb{R}^{B \times 2d_T}$.

Step 2: Self-similarity matrices. For each layer, compute the normalised intra-batch similarity matrices:

$$S_{ij}^{(\ell)} = \text{softmax}(\bar{s}^{(\ell)} \bar{s}^{(\ell)\top})_{ij}, \quad T_{ij}^{(\ell)} = \text{softmax}(\bar{t}^{(\ell)} \bar{t}^{(\ell)\top})_{ij} \quad (14)$$

where softmax is applied row-wise over the B columns, yielding $S^{(\ell)}, T^{(\ell)} \in \mathbb{R}^{B \times B}$.

Step 3: Depth-weighted MSE. The loss is the depth-weighted mean squared error between the two similarity matrices, averaged over layers and batch pairs:

$$\mathcal{L}_{\text{contrastive}} = \frac{1}{B^2 \cdot N} \sum_{\ell=1}^N \frac{\ell}{N} \sum_{i=1}^B \sum_{j=1}^B (S_{ij}^{(\ell)} - T_{ij}^{(\ell)})^2 \quad (15)$$

Note: the contrastive loss operates over the *full patch group* (all patches concatenated), not per micro-batch, so B here is the effective group batch size. All computations are in `float32`.

5.5 Auxiliary Loss Warmup Scale

The auxiliary losses (JEPA and contrastive) are gated by a scalar warmup factor $\gamma \in [0, 1]$ that depends on the current optimiser step s :

$$\gamma(s) = \begin{cases} 0 & s < T_{\text{free}} \\ \frac{s - T_{\text{free}}}{T_{\text{warm}}} & T_{\text{free}} \leq s < T_{\text{free}} + T_{\text{warm}} \\ 1 & s \geq T_{\text{free}} + T_{\text{warm}} \end{cases} \quad (16)$$

where:

$$\begin{aligned} T_{\text{free}} &= \lfloor S_{\text{max}} \times \text{teacher_free_percent} \rfloor \\ T_{\text{warm}} &= \lfloor S_{\text{max}} \times \text{aux_loss_warmup_percent} \rfloor \end{aligned}$$

The effective auxiliary weights are $\tilde{\alpha}_{\text{JEPA}} = \gamma \cdot \alpha_{\text{JEPA}}$ and $\tilde{\alpha}_{\text{contrastive}} = \gamma \cdot \alpha_{\text{contrastive}}$.

5.6 Auxiliary-to-CE Ratio Guard

To prevent instabilities caused by auxiliary losses dominating the CE signal, an adaptive scaling factor $\kappa \in (0, 1]$ is applied to the total weighted auxiliary loss whenever it exceeds a specified fraction $r_{\max}$ of the CE loss magnitude:

$$A = |\tilde{\alpha}_{\text{JEPA}} \mathcal{L}_{\text{JEPA}} + \tilde{\alpha}_{\text{contrastive}} \mathcal{L}_{\text{contrastive}}|, \quad C = |\alpha_{\text{CE}} \mathcal{L}_{\text{CE}}| \quad (17)$$

$$\kappa = \begin{cases} r_{\max} \cdot C / A & A > r_{\max} \cdot C \text{ and } A > 0 \\ 1 & \text{otherwise} \end{cases} \quad (18)$$

Default $r_{\max} = 0.25$ (set `max_aux_to_ce_ratio` ≤ 0 to disable the guard).

5.7 Combined Loss Function

The total scalar loss for a patch group is:

$$\boxed{\mathcal{L} = \alpha_{\text{CE}} \mathcal{L}_{\text{CE}} + \kappa(\tilde{\alpha}_{\text{JEPA}} \mathcal{L}_{\text{JEPA}} + \tilde{\alpha}_{\text{contrastive}} \mathcal{L}_{\text{contrastive}})} \quad (19)$$

Each loss component is accumulated in a weighted sum over the patch group; the weight of patch p with B_p sequences is B_p/B_{total} where $B_{\text{total}} = \sum_p B_p$. CE and JEPA contributions are accumulated per-patch; the contrastive loss is computed once over the full concatenated batch of pooled representations.

6 Training Schedule

6.1 Three-Phase Loss Schedule

Training proceeds through three consecutive phases controlled by `teacher_free_percent` and `aux_loss_warmup_percent`:

1. **Teacher-free phase** (steps $[0, T_{\text{free}})$, default 10% of total): only the CE loss is active ($\gamma = 0$). The teacher is not forwarded, saving compute.
2. **Auxiliary warmup phase** (steps $[T_{\text{free}}, T_{\text{free}} + T_{\text{warm}})$, default next 10%): γ ramps linearly from 0 to 1. The teacher is forwarded and JEPA + contrastive losses are gradually introduced.
3. **Full training phase** (steps $\geq T_{\text{free}} + T_{\text{warm}}$, default remaining 80%): all three losses are active at full weight, subject only to the ratio guard.

6.2 EMA Teacher Transition

At step $\lfloor S_{\max} \times \text{ema_start_percent} \rfloor$ (default 40% of total steps), the `EMATeacherCallback` activates. A deep copy of the current student weights is made, frozen, compiled with `torch.compile`, and attached to the model as `model.ema_teacher`.

Thereafter, at the end of every optimiser step, the EMA teacher parameters θ_T are updated:

$$\theta_T \leftarrow \delta \theta_T + (1 - \delta) \theta_S \quad (20)$$

where $\delta = \text{ema_decay}$ (default 0.999) and θ_S are the current student parameters.

Once the EMA teacher is active, the trainer switches the active teacher from the external frozen model to the EMA copy, and performs a one-time cleanup:

- The teacher projection layers $\{\phi^{(\ell)}\}$ are removed from both the student and the EMA teacher (they are no longer needed since the EMA teacher lives in student-space $\mathbb{R}^{d_s}$).
- The corresponding parameters are removed from the optimiser’s parameter groups.
- The original external teacher model is set to `None` and its GPU memory is freed via `torch.cuda.empty_cache()`.

The EMA teacher is forwarded by calling the student’s own `forward` with $\alpha_{\text{CE}} = \alpha_{\text{JEPa}} = \alpha_{\text{contrastive}} = 0$, retrieving `student_hidden_states` from the output rather than hidden states from the transformer backbone.

6.3 Learning Rate Schedule

The learning rate follows a three-phase piecewise schedule defined as a multiplier on the base learning rate `lr`:

$$\lambda(s) = \begin{cases} \frac{s+1}{W} & 0 \leq s < W \quad (\text{linear warmup, } W = \lceil 0.01 S_{\max} \rceil) \\ 1.0 & W \leq s < W + P \quad (\text{plateau, } P = \lceil 0.49 S_{\max} \rceil) \\ \frac{1}{2} \left(1 + \cos \left(\pi \cdot \frac{s - (W + P) + 1}{C} \right) \right) & W + P \leq s < W + P + C \quad (\text{cosine cooldown, } C = \lceil 0.30 S_{\max} \rceil) \\ 0 & s \geq W + P + C \end{cases} \quad (21)$$

The schedule is implemented as `torch.optim.lr_scheduler.LambdaLR`. The remaining $\approx 20\%$ of steps after the cooldown have zero learning rate.

7 Optimisation

7.1 Standard AdamW Path

When `--muon` is not supplied, all student parameters are optimised with AdamW:

$$\text{AdamW}(\theta, \text{lr} = \text{lr}, \text{weight_decay} = 0) \quad (22)$$

7.2 MuonClip Path

When `--muon` is supplied, parameters are partitioned into three groups:

Group	Selection rule	Optimiser
<code>muon_params</code>	<code>ndim</code> ≥ 2 , name does not contain <code>embed</code> , <code>lm_head</code> , or <code>mlm_head</code>	MuonClip (Muon + C)
<code>adamw_params</code>	embeddings and LM-head weight matrices	AdamW
<code>attention_params</code>	(W_Q, W_K) pairs from every attention layer (QK-Clip targets)	(used inside MuonClip)

The `MuonAdamWWrapper` combines both sub-optimisers behind a single `torch.optim.Optimizer` interface.

7.2.1 Muon (Momentum + Orthogonalisation)

The Muon update for a weight matrix $W \in \mathbb{R}^{m \times n}$ applies Nesterov momentum and then orthogonalises the update via five steps of the Newton–Schulz iteration before scaling by the learning rate:

$$\tilde{G} = \text{NewtonSchulz5}(G_t), \quad W \leftarrow W - \eta_{\text{muon}} \tilde{G} \quad (23)$$

where G_t is the current (momentum-adjusted) gradient and `NewtonSchulz5` maps G_t to a matrix with approximately orthonormal rows (or columns), stabilising the effective learning rate across layers of different widths.

7.2.2 QK-Clip

For each attention layer ℓ and head h , the maximum attention logit magnitude observed during the most recent forward pass is tracked:

$$s_{\max}^{(\ell,h)} = \max_{i,j} \frac{\langle q_i^{(\ell,h)}, k_j^{(\ell,h)} \rangle}{\sqrt{h_d}} \quad (24)$$

where the max is over sequence positions. When `output_s_max = True` (default during training), the model collects these values and passes them back as `DSM2Output.s_max`. The trainer aggregates $s_{\max}$ across patches by taking the element-wise maximum:

$$s_{\max}^{(\ell,h)} \leftarrow \max_p s_{\max,p}^{(\ell,h)} \quad (25)$$

Before the optimiser step, $s_{\max}$ is injected into the `MuonClip` instance. QK-Clip then limits the effective step size on W_Q and W_K to prevent the attention logit magnitude from growing beyond the threshold $\tau = \text{muon_tau}$ (default 100).

7.2.3 Gradient Clipping

When `max_grad_norm > 0`, global gradient clipping is applied to the student model before `optimizer.step()`:

$$g \leftarrow g \cdot \min\left(1, \frac{\text{max_grad_norm}}{\|g\|_2}\right) \quad (26)$$

8 Distributed Training

8.1 Process Group Initialisation

If `world_size > 1` and `--no_init_distributed` is not set, the trainer calls `torch.distributed.init_process_group` with the specified `distributed_backend` (default `gloo`) and `init_method="env://"`. Environment variables `RANK`, `WORLD_SIZE`, and `LOCAL_RANK` are read by `infer_rank_world_size_local_rank`.

8.2 Model Parallelism

After `torch.compile`, the model is wrapped in `DistributedDataParallel` (DDP) on CUDA runs:

`DistributedDataParallel(model, device_ids=[local_rank], output_device=local_rank)`

Gradient synchronisation across ranks is deferred during inner patch-group accumulation steps using `model.no_sync()`. A full `all_reduce` occurs only at the final patch of each gradient accumulation step.

8.3 Distributed Loss Reduction

During evaluation, per-rank losses are reduced via `dist.all_reduce` (sum, then divide by world size). Logits and labels are gathered across ranks with `dist.all_gather_object` and concatenated on the main process (rank 0) before metric computation.

9 Evaluation

9.1 Evaluation Protocol

An initial evaluation on the test set is performed before training begins. During training, evaluation runs on the validation set every `eval_every` optimiser steps (defaulting to `save_every` = 1000 when not specified). A final evaluation on the test set is performed after training completes.

9.2 Metrics

The `ComputeDSM2Metrics` class computes the following metrics from the predicted logits $\hat{Y} \in \mathbb{R}^{B \times L \times V}$ and ground-truth token IDs $Y \in \mathbb{Z}^{B \times L}$ (positions with label -100 are ignored):

Metric	Definition / Notes
Cross-entropy loss	$\mathcal{L}_{CE}$ computed in <code>float32</code> over masked positions
Alignment score	Per-sequence alignment score from <code>GetAlignmentScoreFromLogits</code> , averaged over
Accuracy	Top-1 accuracy $\hat{y}_{ij} = \arg \max_v \hat{Y}_{ijv}$ at masked positions
F1 (weighted)	Weighted-average F1 over residue classes at masked positions
Precision (weighted)	Weighted-average precision at masked positions
Recall (weighted)	Weighted-average recall at masked positions
MCC	Matthews correlation coefficient at masked positions

All classification metrics are computed with `scikit-learn` functions after flattening valid (non- -100) positions.

10 Checkpointing and Logging

10.1 Checkpointing

Every `save_every` optimiser steps, the main process (rank 0) saves:

- The unwrapped student model via `save_pretrained` into `<save_path>/checkpoint-step-<N>/`.
- Optimiser state to `optimizer.pt`.
- Scheduler state to `scheduler.pt`.

At the end of training, when an HF token is provided, the final model is pushed to the HuggingFace Hub at the path specified by `save_path` (private repository).

10.2 Weights & Biases Logging

When both `WANDB_AVAILABLE=true` and `--wandb_token` are provided, the following are logged every `logging_steps` optimiser steps under the `train/` prefix and after each evaluation under `valid/` or `test/` prefixes:

Key	Description
<code>ce_loss</code>	Raw (unweighted) CE loss for the patch group
<code>jepa_loss</code>	Raw JEPA loss
<code>contrastive_loss</code>	Raw contrastive loss
<code>weighted_ce_loss</code>	$\alpha_{\text{CE}} \cdot \mathcal{L}_{\text{CE}}$
<code>weighted_jepa_loss</code>	$\tilde{\alpha}_{\text{JEPA}} \cdot \mathcal{L}_{\text{JEPA}}$
<code>weighted_contrastive_loss</code>	$\tilde{\alpha}_{\text{contrastive}} \cdot \mathcal{L}_{\text{contrastive}}$
<code>weighted_aux_loss_pre_guard</code>	Weighted aux sum before ratio guard
<code>weighted_aux_loss</code>	Weighted aux sum after ratio guard
<code>aux_loss_scale</code>	Current $\gamma(s)$
<code>aux_guard_scale</code>	Current κ

11 Hyperparameters

Table ?? lists every command-line argument, its type, default value, and purpose.

Table 1: All DSM2 training arguments and their defaults.

Argument	Type	Default	Description
<code>--hf_token</code>	str	None	HuggingFace authentication token
<code>--wandb_token</code>	str	None	Weights & Biases API token
<code>--wandb_project</code>	str	DSM2	W&B project name
<code>--save_path</code>	str	GleghornLab/DSM2-600F	HF Hub repo and local checkpoint prefix
<code>--distributed_backend</code>	str	gloo	<code>torch.distributed</code> backend
<code>--no_init_distributed</code>	flag	False	Skip <code>init_process_group</code>
<code>--no_pin_memory</code>	flag	False	Disable pinned-memory DataLoaders
<code>--bugfix</code>	flag	False	Use tiny debug configuration
<code>--teacher_model_path</code>	str	Synthyra/DPLM-3B	HF path for the teacher model
<code>--pretrained_weights</code>	str	None	HF path to seed student weights; skips projection layers if set
<code>--student_hidden_size</code>	int	256	d_s , must satisfy $d_s \bmod 64 = 0$
<code>--student_expansion_ratio</code>	float	4.0	d_{ff}/d_s ratio for the MLP
<code>--sliding_window_size</code>	int	512	Sliding-window size for <code>flex</code> attention
<code>--dilation</code>	int	16	Dilation factor for <code>flex</code> attention
<code>--attn_backend</code>	str	auto	Attention backend (see Section ??)
<code>--data_path</code>	str	Synthyra/uniref50	HuggingFace dataset identifier

Table 1 (continued)

Argument	Type	Default	Description
--sequence_column	str	sequence	Column name containing amino-acid sequences
--max_length	int	2048	Maximum tokenised sequence length
--train_limit	int	0	Max train samples ($\leq 0 = \text{all}$)
--valid_limit	int	1000	Max validation samples
--test_limit	int	1000	Max test samples
--shuffle_seed	int	42	Random seed for dataset shuffling
--dataloader_num_workers	int	0	DataLoader worker processes
--dataloader_prefetch_factor	int	2	Prefetch factor when workers > 0
--lr	float	1e-4	Base learning rate for AdamW
--batch_size	int	64	Effective batch size meta-data; sets <code>patch_accum = [batch_size/patch_size]</code>
--patch_size	int	16	Micro-batch size per forward pass
--grad_accum	int	4	Gradient accumulation steps
--max_steps	int	100000	Total optimiser steps $S_{\max}$
--max_grad_norm	float	0.0	Gradient clipping norm ($0 = \text{disabled}$)
--muon	flag	False	Enable MuonClip optimiser
--muon_lr	float	0.001	Learning rate for the Muon sub-optimiser
--muon_tau	float	100.0	QK-Clip τ threshold
--alpha_ce	float	1.0	Weight α_{CE} on CE loss
--alpha_jepa	float	0.1	Weight α_{JEPA} on JEPA loss
--alpha_contrastive	float	1.0	Weight $\alpha_{\text{contrastive}}$ on contrastive loss
--teacher_free_percent	float	0.1	Fraction of steps in CE-only phase
--aux_loss_warmup_percent	float	0.1	Fraction of steps for aux-loss linear warmup
--max_aux_to_ce_ratio	float	0.25	Cap $r_{\max}$ on aux/CE ratio ($\leq 0 = \text{disabled}$)
--ema_start_percent	float	0.4	Fraction of steps before EMA teacher activates
--ema_decay	float	0.999	EMA decay δ
--save_every	int	1000	Save checkpoint every N optimiser steps
--eval_every	int	0	Evaluate every N steps (≤ 0 uses <code>save_every</code>)
--logging_steps	int	100	Log training metrics every N steps

12 Summary of Training Phases

Figure ?? summarises the interplay of all training phases across the total number of optimiser steps $S_{\max}$ under the default hyperparameter settings.

Step range	% of $S_{\max}$	CE	JEPA + Contrastive	Teacher source
$[0, 0.10 S_{\max})$	10%	✓	×	(not forwarded)
$[0.10, 0.20 S_{\max})$	10%	✓	linear ramp $\gamma \in [0, 1]$	External (frozen)
$[0.20, 0.40 S_{\max})$	20%	✓	✓(full)	External (frozen)
$[0.40, 1.00 S_{\max})$	60%	✓	✓(full)	EMA (student self-distillation)

*tableFigure 1: Training phase schedule under default settings. At step $0.40 S_{\max}$ the EMA teacher activates and the external teacher + projection layers are discarded.